

অপারেটিং সিস্টেমের ভূমিকা

(Introduction to Operating Systems)

প্রফেসর চেস্টার রেবেরো

(Prof. Chester Rebeiro)

কম্পিউটার বিজ্ঞান ও প্রকৌশল বিভাগ

(Department of Computer Science and Engineering),

ইন্ডিয়ান ইনস্টিটিউট অফ টেকনোলজি, মাদ্রাজ

(Indian Institute of Technology, Madras)

সপ্তাহ – 08

বক্তৃতা – 35

অপারেটিং সিস্টেমের নিরাপত্তা

(Operating System Security)

হ্যালো এই ভিডিওতে আমরা অপারেটিং সিস্টেমের security দেখব। এখন, নিরাপত্তা এই বর্তমান বিশ্বের কাছে অত্যন্ত গুরুত্বপূর্ণ হয়ে ওঠে কারণ সিস্টেমগুলি সবসময় সংযুক্ত থাকে, ফলে সিস্টেমটি সন্দেহজনক (malicious) প্রোগ্রামগুলির মধ্যে প্রবেশ করার জন্য এটি খুব সহজ হয়ে যায়। অতএব, অপারেটিং সিস্টেমগুলিকে এমনভাবে design করা উচিত যাতে এই ধরনের malicious অনুপ্রবেশের কারণে data এর কোনও ক্ষতি যতটা সম্ভব প্রতিরোধ করা। সুতরাং, আজকের ভিডিওতে অপারেটিং সিস্টেমের নিরাপত্তা সম্পর্কে সংক্ষিপ্ত পরিচয় দেখতে পাবেন।

(স্লাইড সময় : 01:00 দেখুন)

সুতরাং, যখনই আমরা একটি সিস্টেম ডিজাইন করি, আমাদের কিছু নিরাপত্তা লক্ষ্য আছে এবং এই লক্ষ্যগুলি হল গোপনীয়তা (secrecy), সত্যতা (integrity) এবং সিস্টেমের উপলব্ধতা (availability) আছে। এখন, secrecy মানে কিছু নির্দিষ্ট বস্তু বিশেষ প্রসেসে visible হওয়া উচিত নয়। উদাহরণস্বরূপ একটি প্রসেসের বিশেষাধিকার (privilege) এবং সেই প্রক্রিয়াটি চলাকালীন ব্যবহারকারীর privilege এর উপর নির্ভর করে, যেমন disk এর

নির্দিষ্ট বস্তু বা নির্দিষ্ট network socket, সেই প্রক্রিয়ার দ্বারা পড়া উচিত হবে না। মূলত কি secrecy অর্জন করা হয় যে কিছু বস্তুর অননুমোদিত প্রকাশ (unauthorized disclosure) প্রতিরোধ করা উচিত। কোন secrecy বা confidentiality অর্জন করা হয় যে file গুলি এবং socket গুলি যেমন নির্দিষ্ট বস্তুর অননুমোদিত প্রকাশ করা উচিত নয়। Integrity এর সাথে অননুমোদিত পরিবর্তন অন্য কথায়, একটি malicious user বা একটি নির্দিষ্ট user এর জন্য কোনো বিশেষ বস্তু, যেমন file বা socket হিসাবে লিখতে সক্ষম হওয়া উচিত নয় এমন user। এর একটি উদাহরণ নিতে, সিস্টেমে একটি সাধারণ user কিছু সিস্টেম সম্পর্কিত ফাইলগুলি সংশোধন করতে সক্ষম হবে না। Availability অপরপক্ষে, একটি নির্দিষ্ট user বা একটি নির্দিষ্ট প্রক্রিয়ার system resource এর সীমিত ব্যবহার রয়েছে। অন্য কথায়, একটি নির্দিষ্ট user সম্পূর্ণ system resource ব্যবহার করতে সক্ষম হওয়া উচিত নয়। যদি এই ধরনের কোন ব্যবস্থা সিস্টেমের মধ্যে উপলব্ধ না হয় তবে এটি denial of service attack হিসাবে পরিচিত হয়, যার ফলে সিস্টেমের উপর চালানো কিছু malicious প্রোগ্রাম অন্য যে কোনও প্রোগ্রামকে প্রতিরোধ করবে যা কিছু hardware resource যেমন- CPU , RAM বা disk। একটি সিস্টেমের availability মূলত নির্ভর করে কিভাবে অপারেটিং সিস্টেম ডিজাইন করা হয়। উদাহরণস্বরূপ, আমরা দেখেছি কিভাবে CPU schedulers ডিজাইন করা যায় যাতে সুনির্দিষ্ট সময়সূচী নিশ্চিত করা যায় যে সিস্টেমের প্রতিটি প্রক্রিয়া তার priority এর ভিত্তিতে CPU এর একটি ন্যায্য অংশ পায়। এখন, secrecy এবং integrity অর্জনের জন্য, সাধারণত কোন অপারেটিং সিস্টেমে access control আছে। সুতরাং, আমরা এই বিশেষ ভিডিওতে access control কৌশল দেখব।

(স্লাইড সময় : 04:04 দেখুন)

Access control অ্যাক্সেস কন্ট্রোল অর্জন করার জন্য, এর মানে হল যে প্রতিটি access কোনও ফাইল বা একটি socket এর মত বা একটি printer বা একটি monitor এর মত একটি hardware device হতে পারে। মূলত, যদি একটি প্রক্রিয়া একটি নির্দিষ্ট ফাইল বা সিস্টেমের অন্য কোন বস্তুর access পেতে চায়, এটি একটি access control প্রক্রিয়ার মাধ্যমে যেতে হবে। মূলত, শুধুমাত্র অনুমোদিত প্রবেশাধিকার স্থান নিতে পারেন। অন্য কথায়, যদি একটি প্রক্রিয়া একটি বস্তুকে access করার প্রয়োজন হয়, তাহলে স্পষ্টভাবে এটি নির্দিষ্ট বস্তুর access দেওয়া উচিত। সুতরাং, অ্যাক্সেস মানে কি? মূলত access অ্যাক্সেস দ্বারা আমরা একটি নির্দিষ্ট প্রক্রিয়া যা অপারেশন করতে ইচ্ছুক বুঝি। সুতরাং, এই অপারেশনগুলি write, read, execute, delete হতে পারে। এখন, এই access এর অনুমতি দেওয়া উচিত এবং শুধুমাত্র যদি এই প্রক্রিয়াটি access অ্যাক্সেস করার জন্য নির্দিষ্ট প্রক্রিয়া অনুমোদিত হয়। অন্য কথায়, যদি কোনও একটি নির্দিষ্ট প্রোগ্রামটি চালানোর জন্য অথবা নির্দিষ্ট ফাইলটি পড়তে বা লিখতে কোনও প্রক্রিয়া অনুমোদিত হয় তবেই অপারেশনটির অনুমতি দেওয়া উচিত।

(স্লাইড সময় : 05:39 দেখুন)

Access control system implement করার জন্য, তিনটি উপাদান রয়েছে: নিরাপত্তা নীতি (security policies), নিরাপত্তা মডেল (security model) এবং নিরাপত্তা ব্যবস্থা (security mechanism)। এখন security policy উচ্চ মাত্রার নিয়ম যা access control কে সংজ্ঞায়িত করে। এখন security model, access control model এর একটি mathematical model। মূলত এটি একটি security policy এর একটি mathematical model বা formal model। এবং এই মডেল সিস্টেম সম্পর্কে বিভিন্ন জিনিস প্রমাণ করতে সাহায্য করার জন্য ব্যবহার করা হয়, উদাহরণস্বরূপ একটি নির্দিষ্ট সিস্টেম নিরাপদ প্রমাণ করার জন্য এই বিশেষ security model ব্যবহার করতে পারে। এখন, security ব্যবস্থা নিম্ন স্তরের হার্ডওয়্যার বা সফ্টওয়্যার যা নীতির কার্যকরী বাস্তবায়ন এবং security policy এবং মডেল। সুতরাং, অন্য কথায় আপনি এই উচ্চ পর্যায়ের বিধিনিষেধের কথা মনে করতে পারেন যেগুলি security policy গুলি বা security aspect গুলিকে সংজ্ঞায়িত করে যা সিস্টেম দ্বারা সমর্থিত হতে হবে। এই একটি mathematical model বা সামান্য নিম্ন স্তরের প্রতিনিধিত্ব এবং এই প্রক্রিয়া নীতি বাস্তব বাস্তবায়ন হয়। সুতরাং, আসুন বিস্তারিতভাবে এই প্রতিটি দেখি।

(স্লাইড সময় : 07:10 দেখুন)

সুতরাং, security policy মূলত সিস্টেমের মধ্যে কিছু নীতি প্রয়োগ করার জন্য একটি প্রকল্প। সুতরাং কিভাবে একটি সিস্টেমের কোন পলিসি উচিত তা আমরা কিভাবে সিদ্ধান্ত নিই? সিস্টেম দ্বারা কী কী threat হুমকি উপস্থিত রয়েছে তা দ্বারা সংজ্ঞায়িত করা হয় এবং সিস্টেমটি কীভাবে ডিজাইন করা হয়েছে। সুতরাং, একটি security policy develop করা সহজ নয়। সুতরাং এই ধরনের threat প্রতিরোধ করার জন্য এই threat গুলি সনাক্ত করা গেলে বিভিন্ন ধরনের threat গুলি কি কি সিস্টেমের সম্ভাব্য বিভিন্ন attack গুলি চিহ্নিত করতে পারে এবং একবার এই threat attack জানা গেলে তখন নীতিগুলি তৈরি করা হয় এই সমস্ত attack গুলোকে প্রতিরোধ করার জন্য। সুতরাং, security policy আসলে কেমন দেখাচ্ছে? সুতরাং একটি security policy মূলত statement set সেট হবে। এই statement অত্যন্ত সংক্ষিপ্ত এবং সুনির্দিষ্ট এবং নীতির লক্ষ্য থাকা উচিত। সুতরাং, এই লক্ষ্যগুলি সবাই সম্প্রদায় বা সমগ্র ব্যবস্থার দ্বারা এই ব্যবস্থার develop এর জন্য বা নির্দিষ্ট top level ব্যবস্থাপনা দ্বারা সম্মত হওয়া উচিত এবং এই সুরক্ষা মডেলের ভিত্তি গঠন করে যা নীতির formal mathematical উপস্থাপনা।

(স্লাইড সময় : 08:42 দেখুন)

তাহলে আসুন আমরা কীভাবে একটি security policy লেখা উচিত না তা দেখি। সুতরাং, আমরা এখন যা দেখছি তা হল নিরাপত্তা নীতিগুলি শুধুমাত্র কম্পিউটার সিস্টেমের জন্য নয়, একটি সংস্থার জন্যও হতে পারে। সুতরাং, এই বিশেষ security policy এই বিশেষ কোম্পানীর জন্য বলা হয় Megacorp Incorporated। সুতরাং, সেখানে security policy নিরাপত্তা নীতি খুবই সংক্ষিপ্ত, এর মাত্র চারটি নিম্নরূপ বিবৃতি আছে; এই নীতি

management দ্বারা অনুমোদিত, সমস্ত staff security policy মেনে চলবে, শুধুমাত্র 'need-to-know' দেব data উপলব্ধ হবে, এই নীতির সমস্ত লক্ষ্যন একযোগে security কে রিপোর্ট করা হবে। সুতরাং এই বিশেষ security policy এর সঙ্গে সমস্যা কি? সুতরাং যদি আমরা এই বিশেষ কোম্পানীর কাছ থেকে এই নীতিতে আরো ঘনিষ্ঠভাবে দৃষ্টিপাত করি, তাহলে আমরা দেখতে পাই যে এইগুলির অনেকগুলি ত্রুটি আছে। সুতরাং, মূলত নীতি সম্পূর্ণ না এবং একটি নিরাপত্তা ব্যবস্থা নির্মাণ করতে ব্যবহার করা যাবে না। সুতরাং উদাহরণস্বরূপ, এই নীতি management দ্বারা অনুমোদিত হয় প্রথম জিনিস। সুতরাং, সাধারণত আমাদের security policy এর অনুমোদন নীতি নথির অংশ হওয়া উচিত নয়। দ্বিতীয়ত, সমস্ত staff মান্য করবে। সুতরাং, আমরা এর দ্বারা কি মানে? কে বলবে যে একটি staff মেনে চলবে? আসলে কি নীতিগতভাবে এই নীতির প্রতি আনুগত্য করার জন্য নৈতিকতার প্রয়োজন হয় বা আসলে এমন একটি ইউনিট আছে যা এই বিশেষ নীতিমালার জন্য পালন করতে সংস্থার সমস্ত কর্মীদের বাধ্য করে? তৃতীয়ত, 'need-to-know', তাই কীভাবে বা কী জানতে হবে তা কীভাবে নির্ধারণ করা যায়? কে নির্ধারণ করবে কে জানবে বা কোন তথ্য? সুতরাং, এই নির্দিষ্ট নীতি বিবৃতি থেকে নির্দিষ্ট করা হয় না বা খুব স্পষ্ট নয়। অবশেষে, এই নীতির সমস্ত লক্ষ্যন একযোগে নিরাপত্তা রিপোর্ট করা উচিত। সুতরাং, কীভাবে লক্ষ্যন হয়? নিরাপত্তা পন্থা এই ফর্মটি কিছুই বলে না? কাকে কাকে তাদের রিপোর্ট করতে হয়? তাই এই দুটি এখানে এখানে উল্লিখিত হয় না, আপনি যেমন দেখতে যদিও এই বিশেষ নীতি খুব ছোট এবং মাত্র চার পয়েন্ট সংমিশ্রিত, এখনও এটি অনেক অপব্যবহার বাড়ে এবং এই ধরনের দ্ব্যর্থতা যে কোম্পানির জন্য একটি দুর্বল security policy হবে। সুতরাং, যখন কোম্পানির জন্য একটি নিরাপত্তা নীতি লিখতে হয় এটি সব দিক থেকে সম্পূর্ণ হওয়া উচিত।

(স্লাইড সময় : 11:36 দেখুন)

এখন, আসুন security model টি দেখি। মূলত কেন আমাদের security model থাকা দরকার? কেন আমরা security policy থেকে যেতে পারি না, security policy ব্যবহার করে সরাসরি নিরাপত্তা ব্যবস্থা প্রয়োগ করতে পারি না? কেন এই পুরো অনুক্রমে একটি security model উপস্থাপন করা প্রয়োজন? মূলত একটি security model থাকার মাধ্যমে, আমরা একটি নিম্ন এবং আরো আনুষ্ঠানিক রক্ষণাবেক্ষণ security policy model করতে সক্ষম। সুতরাং এইভাবে নিরাপত্তা নীতিতে কোনও ফাঁক হবে না এবং দ্বিতীয়ত, আমরা যুক্তিগুলি বা প্রমাণ করার জন্য সরঞ্জামগুলি এবং কৌশলগুলি ব্যবহার করতে পারি যাতে সিস্টেম নিশ্চিতভাবে নিরাপত্তার নীতির প্রতি সুরক্ষিত হয় যা নির্ধারণ করা হয়েছিল।

(স্লাইড সময় : 12:31 দেখুন)

সুতরাং, শেষ দিকটি হল নিরাপত্তা ব্যবস্থা (security mechanism)। তাই security mechanism security policy বাস্তবায়ন করে। এখন, এটি গুরুত্বপূর্ণ যে নিরাপত্তা ব্যবস্থা বাস্তবায়ন ক্রটি মুক্ত, এর কোনো ক্রটি নেই। কোন ক্রটি না থাকা এটা অত্যন্ত গুরুত্বপূর্ণ? এটার কারণ security mechanism এ ক্রটি থাকলে, আক্রমণকারীরা সেই ক্রটিগুলি কাজে লাগান এবং সেই সিস্টেমের অননুমোদিত প্রবেশাধিকার পেতে পারে। দ্বিতীয়ত, security mechanism এর বাস্তবায়ন বিশ্বস্ত base হতে হবে। মূলত এই মূল সিস্টেম থেকে নিরাপত্তার উপর নির্ভর করবে যেটাই প্রধান। অতএব, এটি নিজেই ক্রটিপূর্ণ এবং অসম্পূর্ণ যদি, এটি একটি সুরক্ষিত সিস্টেম তৈরি করা হবে না। সুতরাং, নিরাপত্তা ব্যবস্থা বাস্তবায়নের বৈশিষ্ট্য হল এটি tamper proof non by-passable হওয়া উচিত। non by-passable দ্বারা, আমরা বলি যে সিস্টেমের মধ্যে সমস্ত অ্যাক্সেস প্রথমে নিরাপত্তা ব্যবস্থার দ্বারা মূল্যায়ন করা উচিত। তারপর এটি একটি security kernel হওয়া উচিত, এর দ্বারা এর অর্থ কী হয় যে security policy এর সম্পূর্ণ প্রয়োগটি সিস্টেমের একটি সীমিত অংশে সীমাবদ্ধ হওয়া উচিত এবং এটি সিস্টেমের বিভিন্ন অংশে ছড়িয়ে দেওয়া উচিত। সুতরাং, একটি নিরাপত্তা কার্নেল থাকার যেখানে সবকিছুই সীমাবদ্ধ, পুরো সিস্টেমের একটি ছোট অংশটি নিরাপত্তার বিষয়টি পরীক্ষা এবং ডিবাগ ও যাচাই করতে সহজ করে তুলবে। আবার এটি ছোট হওয়া উচিত। সুতরাং, নিরাপত্তা ব্যবস্থার জন্য একটি ছোট আকার নিশ্চিত করবে যে এটি কিছু কঠোর যাচাইয়ের সম্মুখীন হতে পারে।

(স্লাইড সময় : 14:35 দেখুন)

এখন, অ্যাক্সেস কন্ট্রলের জন্য 3 টি কৌশল রয়েছে। যথা DAC, MAC এবং RBAC সুতরাং, DAC হল Discretionary Access Control, MAC হল Mandatory Access Control কন্ট্রোল এবং RBAC হল Role-based Access Control। সুতরাং, আমরা এই বিশেষ বক্তৃতার মধ্যে DAC এবং MAC দেখব।

(স্লাইড সময় : 15:00 দেখুন)

Discretionary Access Control বা DAC একটি অ্যাক্সেস নিয়ন্ত্রণ যা অনুরোধকারীর পরিচয়ের উপর ভিত্তি করে। অনুরোধকারীদের অ্যাক্সেসের নিয়মের একটি সেট থাকবে। সুতরাং, এই অ্যাক্সেসের নিয়মসমূহ মূলত নির্ধারণ করবে যে, অনুরোধকারীদের কী বস্তুগুলি ব্যবহার করতে পারে (বা না পারে), এটিকে কীভাবে অ্যাক্সেস করা যায় এবং কোন বস্তুগুলি read, write, execute বা delete ইত্যাদি করতে পারে। এখন, এই বিভিন্ন বস্তুর জন্য সুবিধা দেওয়া বা নিয়ে নেওয়া system administrator দ্বারা হয়। সুতরাং, user দের যদি তাদের কোনো বিশেষাধিকার থাকে তবে তারা অন্যান্য user দের কাছে এই সুবিধাটি প্রদান করতে পারে। একটি DAC সিস্টেমের একটি খুব সাধারণ উদাহরণ যা Access Matrix Model বলে।

(স্লাইড সময় : 16:05 দেখুন)

সুতরাং, 1971 সালে Butler Lampson দ্বারা Access Matrix Model টি ডিজাইন করা হয়েছিল। এটি বিষয় এবং অবজেক্টের সমন্বয়ে গঠিত, এটি ছাড়াও এই ফর্মের একটি টেবিল রয়েছে যেখানে প্রতিটি সেলের একটি নির্দিষ্ট ক্রিয়া রয়েছে। সুতরাং, সিস্টেমগুলি, সিস্টেমে active element গুলি, যেমন সিস্টেম বা প্রসেসরের user বা তথ্য অনুরোধ করে এমন প্রোগ্রাম। অন্যদিকে বস্তুগুলি passive element উদাহরণস্বরূপ data storage। এখন এই নির্দিষ্ট ম্যাট্রিক্সের প্রতিটি cell এ ক্রিয়াটি নির্দিষ্ট করা হয়েছে। সুতরাং, উদাহরণস্বরূপ বিষয় Ann প্রোগ্রাম 1 এর জন্য access অনুমতিপ্রাপ্ত, তিনি file 2 এর জন্য read and write অনুমতিপ্রাপ্ত, তিনি file 1 এর জন্য read and write অনুমতিপ্রাপ্ত এবং সেই সাথে সে ফাইল 1 এর owner। সুতরাং, যদি আপনি আরও দেখেন, Ann এর ফাইল 3 তে কোনও অনুমতি নেই, সে এটা পড়তে পারবে না এবং লিখতে পারবে না এবং সে আসলে প্রোগ্রাম 1 টি পড়তে বা লিখতে পারে।

(স্লাইড সময় : 17:27 দেখুন)

সুতরাং, এই Access Matrix Model টি প্রতিনিধিত্ব করার জন্য একটি আনুষ্ঠানিক পদ্ধতিতে, আমরা যা করতে পারি তা এই যে matrix A কে সংজ্ঞায়িত করে একটি বিষয় X_{si} এবং X_{oj} যা একটি বস্তু। সুতরাং, এই বিশেষ জিনিস থেকে A একটি বিষয় si কে নির্দেশ করে, উদাহরণস্বরূপ বব এর জন্য একটি বস্তু ফাইল 2। তারপর আমরা এই বিশেষ ম্যাট্রিক্স এর প্রতিটি cell এ অধিকার দিতে পারি। উদাহরণস্বরূপ, অধিকারগুলি এই r_1, r_2 এবং r_k এর মতো একটি সেট থেকে নেওয়া যেতে পারে এবং এই বিশেষ জিনিসটি A এর X_{si}, X_{oj} R এর একটি subset। সুতরাং উদাহরণস্বরূপ, এই উদাহরণের অধিকারগুলি হল own, read, write, execute, ইত্যাদি এবং এইটি সেট R হবে এবং এই ম্যাট্রিক্স এর প্রতিটি cell সংশ্লিষ্ট নির্ধারণ করা যায়। সুতরাং, এটা সংজ্ঞায়িত করবে কি subject si object oj এর উপর আছে। এই ছাড়াও primitive operation হিসাবে পরিচিত কিছু আছে। সুতরাং, এইটি O দ্বারা প্রতিনিধিত্ব করা হয় এবং এটি 6 টি অপারেশনের একটি সেট। সুতরাং উদাহরণস্বরূপ, ম্যাট্রিক্সের কিছু স্থানে r লিখুন বা সেই অবস্থান থেকে r মুছে দিন, এখানে r generic rights থেকে নেওয়া হয়েছে। সুতরাং, এটি R এর element । অনুরূপভাবে অন্য অপারেশনগুলি একটি নির্দিষ্ট subject X_s , একটি বস্তু X_o তৈরি করা হয়, বা একটি subject X_s , একটি বস্তু X_o ধ্বংস করে। সুতরাং এইটি একমাত্র primitive operation যে এই বিশেষ access matrix করা যেতে পারে। সুতরাং, আমরা দেখতে পাচ্ছি যে এই primitive অপারেশনগুলির প্রতিটি ম্যাট্রিক্সের একটি নির্দিষ্ট cell এর বিষয়বস্তু সংশোধন করবে বা এটি একটি সম্পূর্ণ column মুছে ফেলতে পারে এবং এটি ধ্বংস করতে পারে। সুতরাং এটি একটি নির্দিষ্ট column বা একটি নির্দিষ্ট row মুছে ফেলতে পারে বা যদি এটি চায় যে এটি একটি নতুন column তৈরি করতে পারে যা একটি subject তৈরি করেছে বা একটি object তৈরি করেছে। তাই একবার, আমরা যেমন primitive অপারেশন সংজ্ঞায়িত

করেছি, আমরা কিছু সংজ্ঞায়িত করতে পারেন যা আরও জটিল এবং এই আদিম অপারেশনগুলির উপর paste করে যা command হিসাবে পরিচিত হয়।

(স্লাইড সময় : 20:17 দেখুন)

সুতরাং, আমাদের কাছে access matrix টি মনে রাখা উচিত যা A দ্বারা নির্দিষ্ট করা আছে, আমাদের কাছে r1 থেকে rk এর generic right আছে এবং আমরা এখানে নির্দিষ্ট হিসাবে আমরা ছয়টি primitive অপারেশন করেছি। সুতরাং, আমরা এইভাবে জটিল কমান্ডগুলি তৈরি করতে এই সবগুলি ব্যবহার করতে পারি। একটি আদর্শ কমান্ড এই মত দেখতে এবং এটি বলে বলা হবে, উদাহরণস্বরূপ আলফা যা কিছু পরামিতি x1 থেকে xn লাগে এবং আমাদের নিয়মের একটি সেট থাকতে পারে। উদাহরণস্বরূপ, যদি R1 এই ম্যাট্রিক্স সেল এবং R2, R3 এবং আরও অনেক কিছুতে থাকে, তাহলে নিম্নলিখিত অপারেশনগুলি অনুসরণ করুন। সুতরাং, আমরা যা বলছি তা হল এই low level generic operations, generic rights অধিকার এবং access matrix এর উপর ভিত্তি করে, আমরা access matrix এ আরো জটিল কমান্ডগুলি করতে পারি।

(স্লাইড সময় : 21:19 দেখুন)

সুতরাং আসুন কিছু কমান্ডের উদাহরণ গ্রহণ করি, উদাহরণস্বরূপ প্রক্রিয়া comma file তৈরি করুন। সুতরাং, এখানে প্রক্রিয়াটি subject এবং এটি বলতে কি বলতে হয়, এই প্রক্রিয়াটি এই ফাইলটি তৈরি করতে চায়। সুতরাং, কমান্ড এই মত কাজ করবে, তাই প্রথম আমরা primitive operation ব্যবহার করি, একটি object ফাইল তৈরি করি এবং তারপর যে প্রক্রিয়া comma file এর মধ্যে লেখে। সুতরাং, এই প্রক্রিয়া comma file access matrix এর cell এর অনুরূপ। সুতরাং এই নির্দিষ্ট কমান্ড উপস্থিত না থাকলে, একটি প্রক্রিয়া আসলে একটি নির্দিষ্ট ফাইল তৈরি করা সম্ভব নয়। সুতরাং, এটি একটি খুব সহজ উদাহরণ। সুতরাং কিছু আরো জটিল হতে পারে এই বিশেষ উদাহরণের মত CONFERr – owner, friend, file । সুতরাং, owner, friend হল subject এবং ফাইল object। এবং access matrix এ এই বিশেষ কমান্ড দ্বারা কি উদ্দেশ্যে করা হয় যে, আমরা অধিকার প্রদান করতে চাই 'r' যা নির্দিষ্ট বস্তুর জন্য এখানে friend এর জন্য আছে। সুতরাং, এই owner এই নির্দিষ্ট ফাইলের সাথে friend এর জন্য right r প্রদান করতে চায়। সুতরাং, কি এই বিশেষ কমান্ড মধ্যে যায়? সুতরাং মূলত আমরা প্রথম পরীক্ষা করি owner প্রকৃতপক্ষে যে ফাইল owner কিনা। সুতরাং, আমরা প্রথম দেখতে পারি যে যদি মালিকের বিষয় সংশ্লিষ্ট ম্যাট্রিক্স cell এর নিজস্ব comma file উপস্থিত থাকে এবং object টি file ফাইল তবে এটি সত্যই r তবে লিখুন যে friend comma file ফাইলের right 'r' লিখুন ম্যাট্রিক্স অংশে। একইভাবে, আমাদের আরেকটি উদাহরণ থাকতে পারে, যেখানে আমরা একটি ex friend থেকে একটি right সরিয়ে ফেলতে পারি। আমরা এই বিশেষ কমান্ড প্রেরণ করেছি, তিনটি parameter; owner, ex friend এবং ফাইল। সুতরাং, মূলত আমরা যা করতে চাই তা হল এই ex friend থেকে right 'r' প্রত্যাহার করতে যা অন্য subject। সুতরাং, কমান্ডটি এইরকম

দেখায়, যদি owner comma file ফাইলটিতে থাকে তবে owner যদি প্রকৃতপক্ষে ফাইলের owner হন এবং right 'r' ex friend comma file উপস্থিত থাকে তাহলে ex friend comma file থেকে 'r' মুছে দিন। অতএব, আমরা right 'r' কে সরিয়ে ফেলতে পারি যা ফাইলটি পড়তে বা লিখতে পারি বা ঠিক করতে পারি, ফাইলটির সাথে ex friend সাপেক্ষে। মূলতঃ ex friend subject এর নির্দিষ্ট ফাইলটিতে right 'r' নেই।

(স্লাইড সময় : 24:24 দেখুন)

সুতরাং একটি protection ব্যবস্থা, যদি আমরা প্রকৃতপক্ষে এই নির্দিষ্ট ম্যাট্রিক্স দ্বারা প্রতিনিধিত্ব করি এবং আমরা যা দেখেছি তা হল আমরা এটিতে কমান্ড প্রয়োগ করতে পারি যেমন আমরা পূর্ববর্তী স্লাইডের উদাহরণগুলিতে কয়েকটি কমান্ড দেখেছি এবং প্রতিটি কমান্ড দিয়ে আমরা এই বিশেষ access matrix সংশোধন করা হয়। সুতরাং, উদাহরণস্বরূপ এই কমান্ডটি এখানে access matrix এ এই cell টি সংশোধন করেছে। অনুরূপভাবে, যদি আপনি অন্য কমান্ড চালান, access matrix এর অন্য অংশ অথবা access matrix এর একাধিক cell সংশোধন করা হবে। সুতরাং, এর উপর ভিত্তি করে আমরা সংজ্ঞায়িত করি, যা Leaky state হিসাবে পরিচিত। Access matrix একটি state right 'r' কে leak বলে অভিহিত করা হয় যদি প্রবেশের ম্যাট্রিক্সের একটি entry তে right ডান 'r' যোগ করে এমন একটি কমান্ড থাকে যা আগে 'r' থাকেনি। সুতরাং, এর মানে হল যে আমাদের এখানে ম্যাট্রিক্সের অবস্থা আছে এবং আমরা একটি কমান্ড চালাচ্ছি এবং এর ফলে right 'r' আছে যা এই বিশেষ ম্যাট্রিক্সে প্রবেশ করে। সুতরাং, আমরা কি বলছি যে ম্যাট্রিক্স state r কে leak করে। এখন, একটি leaky state সবসময় খারাপ হবে না। সুতরাং, এটি ম্যাট্রিক্সের একটি নির্দিষ্ট cell থেকে এই 'r' leaking সিস্টেমের নিরাপত্তার জন্য ভাল বা খারাপ হয় কিনা তা নির্ধারণ করতে হবে।

(স্লাইড সময় : 26:32 দেখুন)

তাই আসুন আমরা কখন সিস্টেমকে safe বলে মনে করি তা সংজ্ঞায়িত করি। সুতরাং, safety এর দুটি সংজ্ঞা আছে। প্রথম সংজ্ঞা; একটি সিস্টেম safe হয় যদি owner এর সম্মতি ছাড়া একটি বস্তুর access অসম্ভব হয়। দ্বিতীয়ত, একজন user যদি কোনও নির্দিষ্ট subject কে অধিকার প্রদান করে এবং কোনও বস্তুর সাপেক্ষে right টি আরো leak হয়।

(স্লাইড সময় : 26:43 দেখুন)

সুতরাং আসুন একটি প্রথাগত মডেল থেকে এই অর্থ কি। ধরুন, একটি subject s অন্য একটি subject s prime কে object o এর right r দেবার পরিকল্পনা করছে। s prime comma o তে r right যেটি ম্যাট্রিক্সের এই বিশেষ কক্ষের সাথে যুক্ত রয়েছে। এটা কি সম্ভব যে পরবর্তীতে ম্যাট্রিক্স এতে অন্য কোথাও প্রবেশ করা যাবে, যদি এমন কিছু সম্ভব হয় তবে সিস্টেমটি unsafe হতে চলেছে। সুতরাং, মূলত একটি সিস্টেম unprotected যদি কোন

অপারেশন বা access matrix চালানো কোন কমান্ড যে বিশেষ অধিকার প্রদান অন্য কারো বা অন্য কোনও ম্যাট্রিক্স এ স্থানান্তর হয়।

(স্লাইড সময় : 27:47 দেখুন)

তাই আমরা একটি unsafe অবস্থার একটি উদাহরণ দেখি। সুতরাং এই দুটি কমান্ড বিবেচনা করা যাক, এক CONFERexecute। সুতরাং subject s subject s prime কে, object O এর জন্য execute right প্রদান করতে চায়। সুতরাং, এই কমান্ডটি CONFERexecute s comma s prime comma O নির্দেশ করে যে কোন subject যদি object এর owner হয় তাহলে এটি সঠিকভাবে right 'x' অন্য subject s prime কে object O execute করার জন্য দিতে পারে। এখন, আসুন আমরা আমাদের সিস্টেমে এই বিশেষ কমান্ডটি বলি যে MODIFY_RIGHT(S,O), যা নিম্নোক্ত হিসাবে উল্লিখিত হয়, যদি কোন নির্দিষ্ট subject কোন object এর execute right থাকে তবে এটি সেই নির্দিষ্ট object এর write enter করতে পারে। অন্য কথায়, এই MODIFY_RIGHT কি করতে পারবে তা হল যদি কোন নির্দিষ্ট subject কোন object execute পারে তবে object টিকে পরিবর্তন করার জন্য এটি আসলে তার right সংশোধন করতে পারে। সুতরাং, মূলত এটি যে বস্তুর মধ্যে লিখতে পারে। সুতরাং আসুন দেখি একটি বিশেষ scenario, কিভাবে এই উদাহরণ একটি unsafe state দেখায়। সুতরাং, আসুন আমরা বলি, বব একটি application object তৈরি করে, সে এটি অন্যদের দ্বারা execute করতে চায় কিন্তু তাদের দ্বারা পরিবর্তিত হবে না। সুতরাং, security ব্যবস্থায় MODIFY_RIGHT উপস্থিতির কারণে সিস্টেমটি স্পষ্টত unsafe। অ্যালিস, অন্য কোন ব্যক্তি যিনি নির্দিষ্ট বস্তুর বা অ্যাপ্লিকেশনের জন্য execute right টি প্রয়োগ করতে পারেন MODIFY_RIGHT কে সেই অ্যাপ্লিকেশনটির পরিবর্তনের অধিকার পেতে এবং একবার এলিস যে অ্যাপ্লিকেশনটি সংশোধন করার অধিকার পায় তা পরে এলিসকে আটকানোর কিছু নেই, আসলে সে অপারেশনটি পরিবর্তন করতে পারে। সুতরাং, আমরা যা দেখি তা হল যে বব একটি নির্দিষ্ট বস্তু চালানোর অনুমোদন দিয়েছেন এবং সিস্টেম policy গুলি বা system access control পদ্ধতি এইভাবে নির্মিত হয়েছে যে এই অধিকার একটি নির্দিষ্ট বস্তুতে উপস্থিত থাকলে, তারপর w right নির্দিষ্ট বস্তুর দেওয়া হয়। মূলত একটি বিষয় একটি বস্তুর জন্য execute অধিকার আছে যদি মূলত যে রূপান্তরিত এবং স্থানান্তরিত হয় এবং এটি w right যে subject সম্পর্কিত যে নির্দিষ্ট object এর সাথে সংযুক্ত হতে হবে। অতএব, সিস্টেমের এই বিশেষ অবস্থা একটি unsafe অবস্থা।

(স্লাইড সময় : 30:48 দেখুন)

তাহলে কিভাবে আমরা আসলে অ্যাক্সেস ম্যাট্রিক্স মডেল বাস্তবায়ন করব? সুতরাং, আমরা দেখেছি যে একটি উপায় হল ম্যাট্রিক্স প্রতিনিধিত্ব। যাইহোক, এই বিশেষ প্রতিনিধিত্বের সঙ্গে সমস্যা এটি খুব sparse এবং খুব বড় হয়। সুতরাং, এটি ব্যবহার করার একটি খুব কার্যকর উপায় নয়। অন্য কৌশলটি এখানে একটি Authorization table হিসাবে পরিচিত কিছু ব্যবহার করা হয় যেমন দেখানো হয়েছে। তাই এই বিশেষ টেবিল database এর উপর ভিত্তি করে এবং এটি আসলে কি আছে যে প্রতিটি row এ একটি user অন্তর্ভুক্ত হয়, এটি একটি right আছে এবং একটি object আছে। তাই, এটি বলছে যে, অ্যাক্সেসের একটি অবজেক্ট ফাইল 1 এ read, write আছে। সুতরাং, অ্যাক্সেসের যে অধিকার আছে তার জন্য এই বিশেষ টেবিলে একটি পৃথক row থাকবে। সুতরাং, এই বিশেষ scenario এর সঙ্গে সমস্যা হয় যে বা এই বিশেষ বাস্তবায়নে, একটি নির্দিষ্ট user একটি নির্দিষ্ট object এ একটি বিশেষ right আছে কিনা তা চিহ্নিত করার জন্য সম্পূর্ণ টেবিল অনুসন্ধানের প্রয়োজন। সুতরাং, এটি একটি বিষয় এবং বস্তুর জন্য একটি নির্দিষ্ট অধিকার নির্ধারণ করতে কতক্ষণ লাগে তার অনেক কর্মক্ষমতা সমস্যা হয়।

(স্লাইড সময় : 32:14 দেখুন)

তাই access matrix model প্রয়োগ করার অন্য একটি কৌশল ক্ষমতা (capability) ব্যবহার করে হয়। এখানে, প্রতিটি user একটি capability তালিকার সঙ্গে যুক্ত করা হয়, প্রতিটি বস্তুর জন্য অনুমোদিত অপারেশন দিয়ে। উদাহরণস্বরূপ, অ্যান, ফাইল 1 এর জন্য এখানে নিম্নলিখিত অধিকার রয়েছে, ফাইল 2 এর জন্য তার কাছে এই অধিকার আছে, প্রোগ্রাম 1 এর জন্য তার এই অধিকার রয়েছে একইভাবে, প্রতিটি user এর জন্য আমাদের এইরকম তালিকা আছে প্রতিটি বস্তুর সাপেক্ষে সেই ব্যবহারকারীর। সুতরাং একটি মডেল বা এই ধরনের একটি বাস্তবায়নের একটি সুবিধা হল একটি distributed system scenario থেকে, কারণ এটি একটি subject বারংবার authentication করতে বাধা দেয়। সুতরাং, মূলত একবার বিষয়টি distributed পরিবেশে নিজেই অনুমোদন করে, তারপর এটি এই সমগ্র তালিকাটি অর্জন করতে পারে এবং সিস্টেমটি প্রতিটি বস্তুর জন্য তার বর্তমান right গুলি জানতে পারবে। যাইহোক, সীমাবদ্ধতা হল যে কোনও right বা একটি তালিকা একটি attacker বা আক্রমণকারী দ্বারা copy করা হয় তাহলে এটি attacker ভেদ্য হয়, তাহলে অপব্যবহার করা যেতে পারে।

(স্লাইড সময় : 33:34 দেখুন)

Access Control Model টি বাস্তবায়নের তৃতীয় উপায় হলো ACL বা Access Control List। মূলত, প্রতিটি object এখানে একটি তালিকার সঙ্গে যুক্ত করা হয় যা প্রতিটি subject সঞ্চালন করতে পারে। মূলত, এটি পূর্বের পথের বিপরীত। সুতরাং, আমাদের প্রতিটি object এর সাথে সংশ্লিষ্ট একটি তালিকা থাকে এবং তালিকাতে প্রতিটি

node একটি subject এবং সংশ্লিষ্ট অপারেশন এখানে উপস্থিত আছে। সুতরাং, এটিতে সুবিধা হল যে এটি ছোট কারণ দ্বারা প্রতিনিধিত্ব করা সহজ। সুতরাং, যদি আপনি দেখেন কিভাবে UNIX আসলে এই ACL প্রয়োগ করে।

(স্লাইড সময় : 34:19 দেখুন)

সুতরাং, আমরা দেখতে পাই যে UNIX সিস্টেমের প্রতিটি ফাইলের সাথে 9 টি বিট সংযুক্ত রয়েছে। সুতরাং, এই প্রতিটি ফাইলের জন্য নির্দিষ্ট করার জন্য authorization দেয়। উদাহরণস্বরূপ ফাইলটির owner এর জন্য 3 টি বিট ফাইলের জন্য read, write, execute নির্ধারণ করে। তারপর group আছে 3 টি বিট r, w, x এবং বাকিগুলো বিশ্বের অন্যদের জন্য, আরো 3 টি বিট আছে অন্যদের জন্য। সুতরাং access matrix এর ACL বাস্তবায়ন ব্যবহার করে একটি UNIX সিস্টেমের মধ্যে, 9 টি বিট জড়িত আছে এবং এই বিট প্রতিটি যে ফাইলের বিভিন্ন ব্যবহারকারীদের অনুমতি দেবে।

(স্লাইড সময় : 35:13 দেখুন)

এখন, discretionary policy এর দুর্বলতা হল যে এটি Trojan Horse আক্রমণগুলির নিয়ন্ত্রণাধীন। তাই Trojan Horse দ্বারা বোঝানো হয় যে এটা একটি বড় দূষিত নয় এমন code এ উপস্থিত একটি ছোট কোড। সুতরাং, বৃহত্তর কোডে উপস্থিত ক্ষুদ্র কোডটি হল ক্ষতিকারক কোড। তাই এই সমস্যাটি হল যে Trojan প্রকৃতপক্ষে একটি virus যা সত্যিই অনেক বড় প্রোগ্রামের সাথে যুক্ত হয়, তাই Trojan আসলে সমস্ত user এর বা সমস্ত প্রসেসের বিশেষাধিকার অর্জন করতে পারে। সুতরাং উদাহরণস্বরূপ যদি আপনি একটি Trojan যা সংযুক্ত একটি প্রোগ্রাম যা চালানো একটি super user right আছে বা একটি route হিসাবে চালানো প্রয়োজন, তাহলে, Trojan নিজেই সব রুট অনুমতি নিতে সক্ষম হবে। এই কারণটি ঘটেছে যে প্রক্রিয়ায় উপস্থিত Trojan বৈধ user এর পক্ষ থেকে অপারেটিং সিস্টেমের অনুরোধ পাঠাবে। তাই অপারেটিং সিস্টেমটি এমন কোনও পার্থক্য করতে সক্ষম হবে না যা কিনা বৈধ প্রক্রিয়ায় রয়েছে, যা একটি বৈধ user দ্বারা শুরু হয় বা কিনা অনুরোধটি Trojan থেকে আসছে। অতএব, সনাক্ত করা কঠিন হয়ে ওঠে।

(স্লাইড সময় : 36:51 দেখুন)

Discretionary policy এর আরেকটি দুর্বলতা হল এটি information flow এর সাথে সম্পর্কিত নয়; information access সহ যে কেউ information প্রচার করতে পারেন। উদাহরণস্বরূপ, আমাদের এখানে এমন এক ব্যক্তি আছেন যিনি একটি নির্দিষ্ট ফাইল পড়তে সক্ষম, প্রকৃতপক্ষে এই ফাইলটির একটি অনুলিপি তৈরি করে এবং এটি কয়েকশ ভিন্ন ব্যক্তিদের স্থানান্তর করার জন্য এই ব্যক্তিটিকে আটকে রাখা যায় না। তাই যেমন discretionary policy গুলি এক ব্যক্তি বা এক user থেকে information flow এর তথ্য প্রবাহের যত্ন করে না। তাই শুধুমাত্র information access প্রতিরোধ করতে সক্ষম, কিন্তু information flow নয়। অন্যদিকে,

information flow এর বিবেচনা করার জন্য, information flow policies হিসাবে পরিচিত যা ব্যবহার করা হয় সেগুলি ব্যবহার করতে হবে যা কীভাবে বিষয়গুলি এবং বস্তুর মধ্যে information flow কে সীমিত করে। তাই, আমরা পরবর্তী ভিডিওতে information flow policy গুলি দেখব। ধন্যবাদ।